

TERRAFORM TASK -05

1. Execute the script shown in the video.

Terraform Remote state and state locking: We can store terraform configuration files and state file in github or any other repository but it is not good practise.

We use s3, terraform storage, hashicorp consul to store the state file.

State locking is used to lock the state file so that no two users can execute the state file at the same point of time.

Remote backend and state locking: We can use s3 as remote backend and dynamo db for state locking.

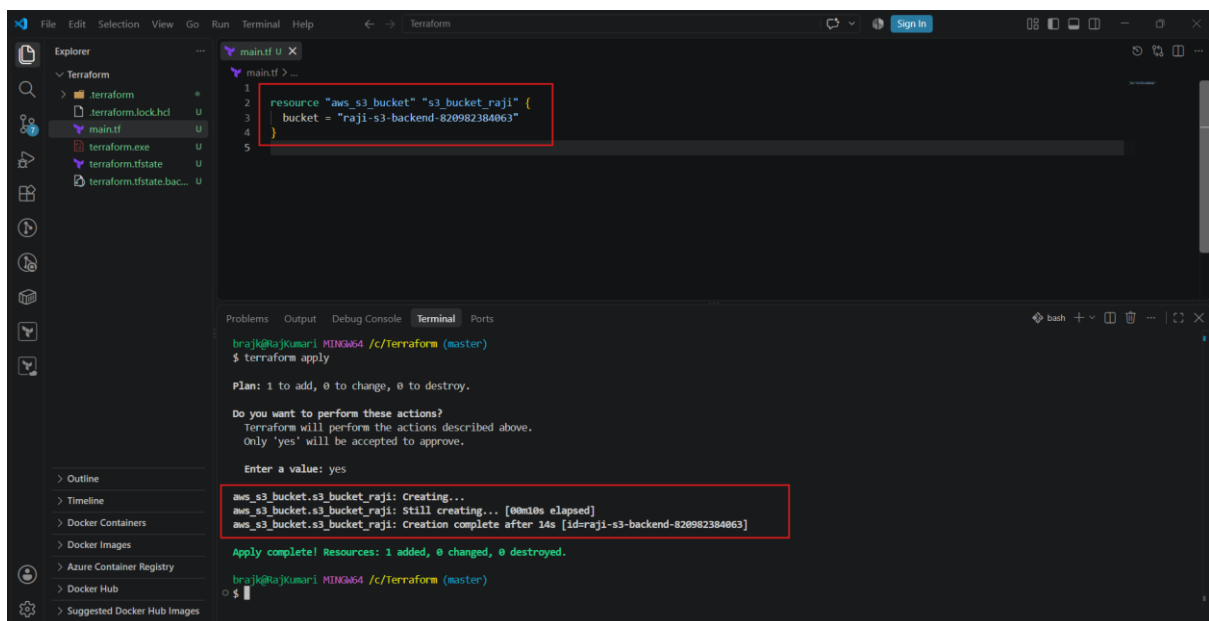
Create s3 using terraform:

```
resource "aws_s3_bucket" "s3_bucket" {
```

```
    bucket = "s3backend"
```

```
    acl = "private"
```

```
}
```



The screenshot shows a VS Code editor with a file explorer on the left containing files like `terraform.lock.hcl`, `main.tf`, `terraform.exe`, `terraform.tfstate`, and `terraform.tfstate.backup`. The main editor displays a Terraform configuration in `main.tf`:

```
1 resource "aws_s3_bucket" "s3_bucket_raji" {
2   bucket = "raji-s3-backend-820982384063"
3 }
4
5
```

Below the editor, the terminal window shows the execution of `terraform apply`. It displays the plan, a confirmation prompt, and the successful creation of the S3 bucket:

```
brajk@rajakumari-MINGW64 /c/Terraform (master)
$ terraform apply

Plan: 1 to add, 0 to change, 0 to destroy.

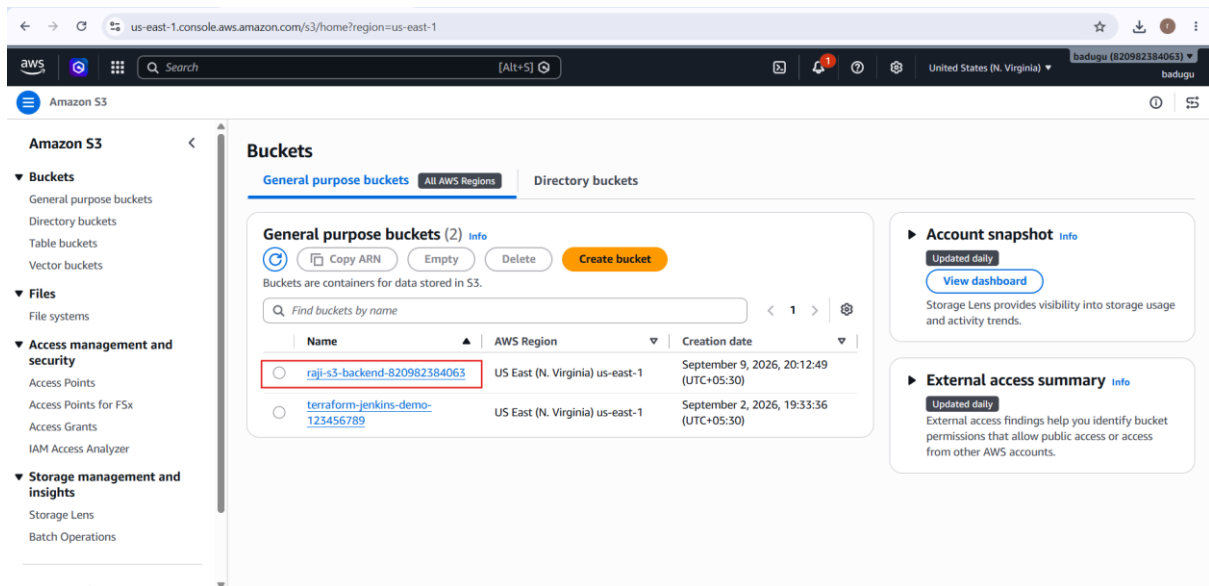
Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

Enter a value: yes

aws_s3_bucket.s3_bucket_raji: Creating...
aws_s3_bucket.s3_bucket_raji: Still creating... [90m10s elapsed]
aws_s3_bucket.s3_bucket_raji: Creation complete after 14s [id=raji-s3-backend-820982384063]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

brajk@rajakumari-MINGW64 /c/Terraform (master)
$
```



Create dynamo db using terraform:

```
resource "aws_dynamodb_table" "dynamodb-terraform-state-lock" {  
  
    name = "terraform-state-lock-dynamo"  
  
    hash_key = "LockID"  
  
    read_capacity = 20  
  
    write_capacity = 20  
  
    attribute {  
  
        name = "LockID"  
  
        type = "S"  
  
    }  
  
}
```

The image shows a VS Code editor with a Terraform configuration file named `main.tf`. The configuration defines an AWS DynamoDB table resource `aws_dynamodb_table.dynamodb-terraform-state-lock` with the following attributes:

```
resource "aws_dynamodb_table" "dynamodb-terraform-state-lock" {
  name         = "terraform-state-lock-dynamo"
  hash_key     = "LockID"
  read_capacity = 20
  write_capacity = 20

  attribute {
    name = "LockID"
    type = "S"
  }
}
```

The terminal output shows the execution of `terraform apply` and the successful creation of the table:

```
braj@braj-kumari-MINGW64 /c/Terraform (master)
$ terraform apply
+ tt1 (known after apply)
}
+ warm_throughput (known after apply)
}

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

aws_dynamodb_table.dynamodb-terraform-state-lock: Creating...
aws_dynamodb_table.dynamodb-terraform-state-lock: Still creating... [00m10s elapsed]
aws_dynamodb_table.dynamodb-terraform-state-lock: Creation complete after 11s [id=terraform-state-lock-dynamo]

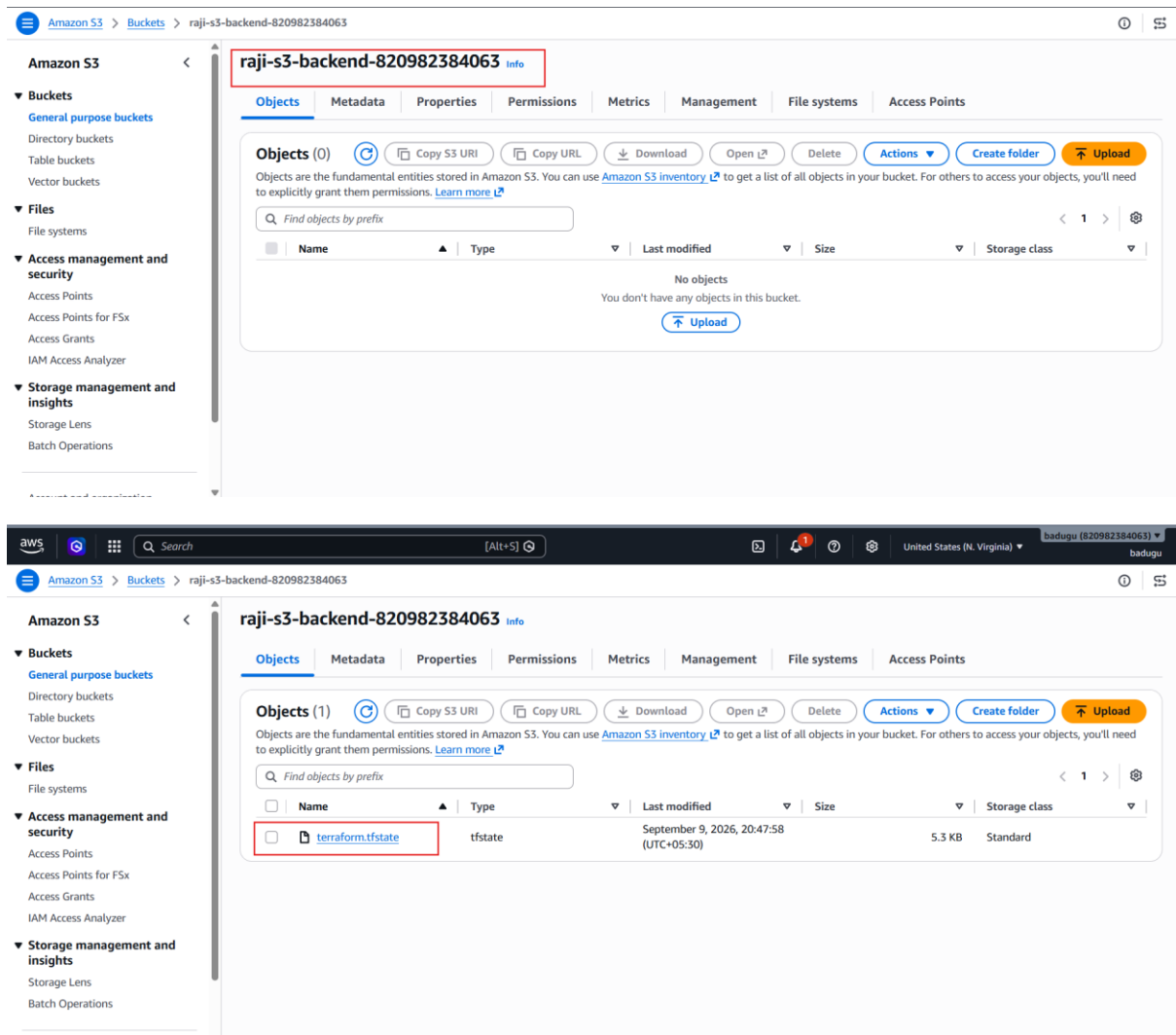
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

The image shows the AWS Management Console, specifically the DynamoDB console. The table `terraform-state-lock-dynamo` is listed with the following details:

Name	Status	Partition key	Sort key	Indexes	Replication Regions	Deletion protection	Favorite	Read capi
terraform-state-lock-dynamo	Active	LockID (S)	-	0	0	Off	☆	Provisione

S3 as backend for terraform.tfstate file:

```
terraform {
  backend "s3" {
    bucket = "s3backend"
    dynamodb_table = "terraform-state-lock-dynamo"
    key = "terraform.tfstate"
    region = "us-east-1"
  }
}
```

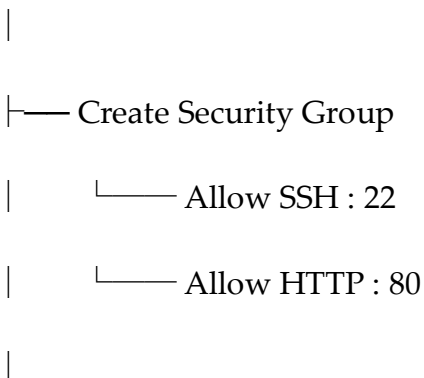


2. Create one EC2 instance with **httpd** installed using a Terraform script

Terraform will create **one EC2 instance**, install **Apache HTTP Server (httpd)**, start it, and allow HTTP traffic so you can open the web page in your browser.

Architecture

Terraform



└── Create EC2 Instance

|

└── User Data

└── Install httpd

└── Start httpd

└── Enable httpd

|

↓

Apache Web Server

|

Port 80

|

Browser

Step 1: Create a Terraform folder

On your Windows PC, create a folder:

ec2-httpd-terraform

Open this folder in VS Code.

You can create these files:

ec2-httpd-terraform/

└── provider.tf

└── main.tf

└── variables.tf

└── outputs.tf

For a beginner, we can also do everything in main.tf, but separating the files is better for learning.

Step 2: Create provider.tf

Put this in provider.tf:

```
terraform {  
  required_providers {  
    aws = {  
      source = "hashicorp/aws"  
      version = "~> 6.0"  
    }  
  }  
}
```

```
provider "aws" {  
  region = "us-east-1"  
}
```

Explanation

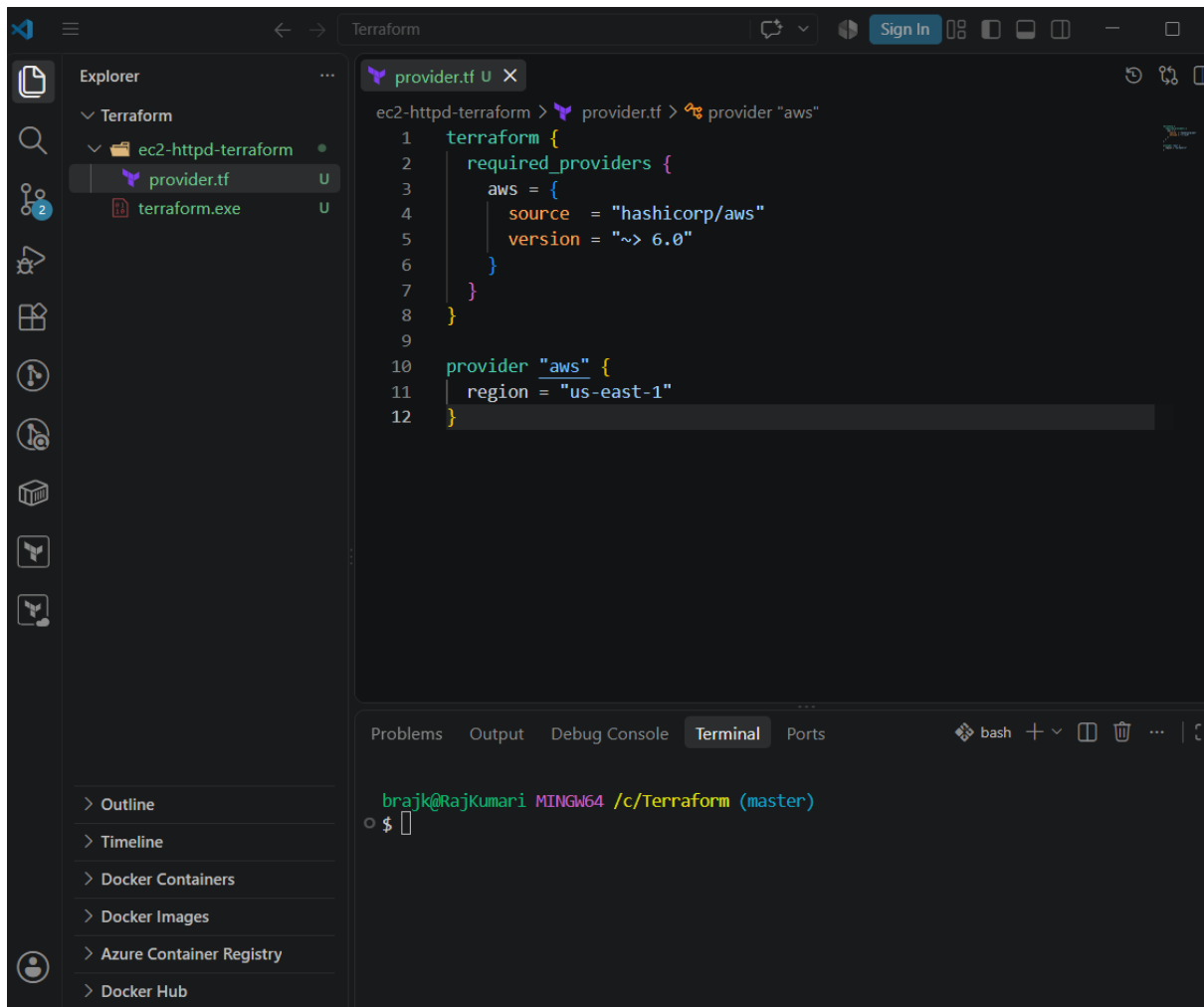
provider "aws"

tells Terraform that we are using AWS.

region = "us-east-1"

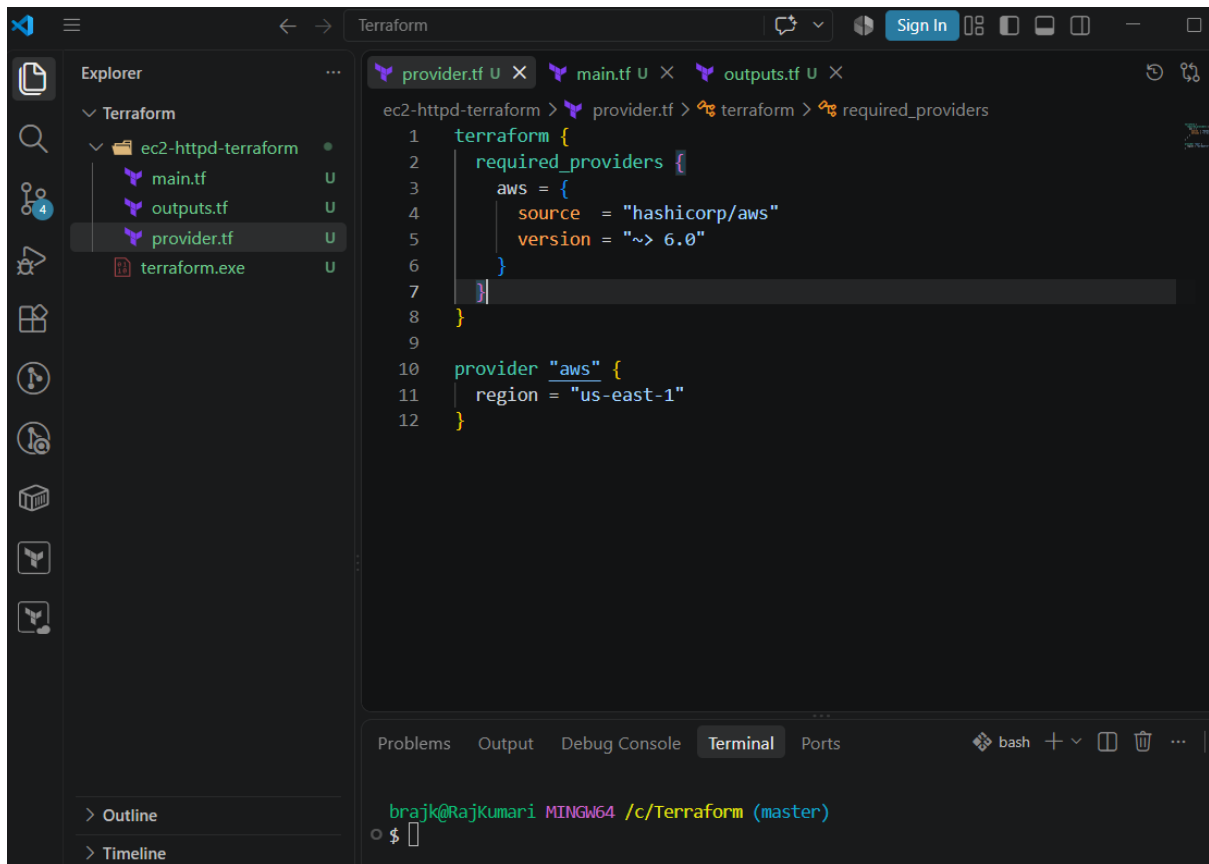
means Terraform will create the resources in **US East (N. Virginia)**.

You can change this if your AWS resources are in another region.



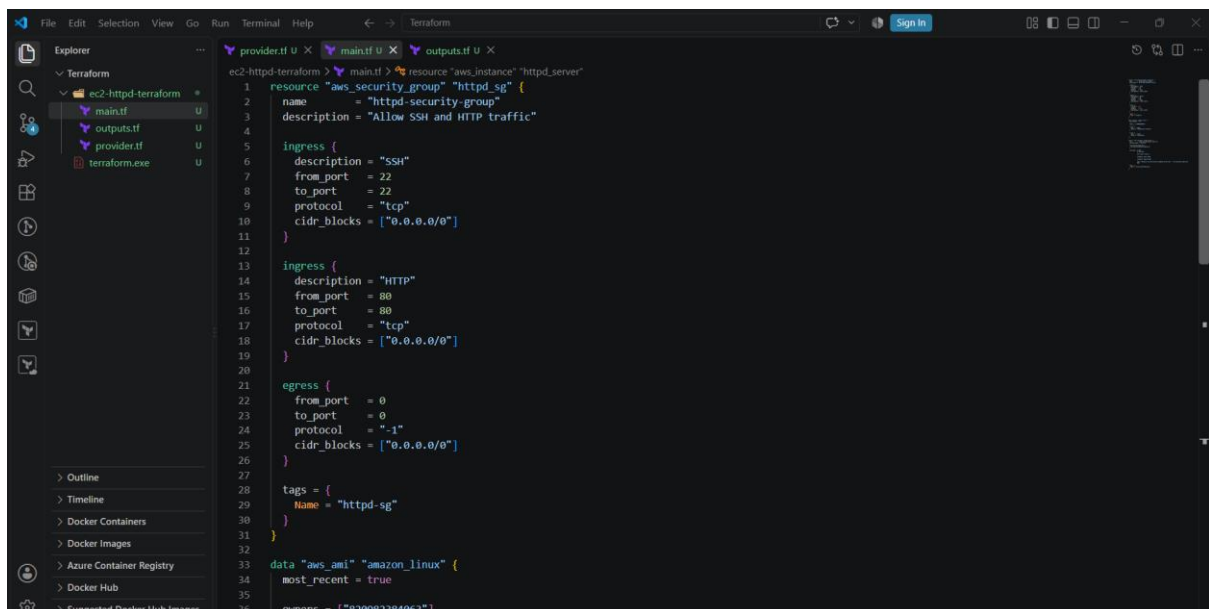
Step 2: Create provider.tf

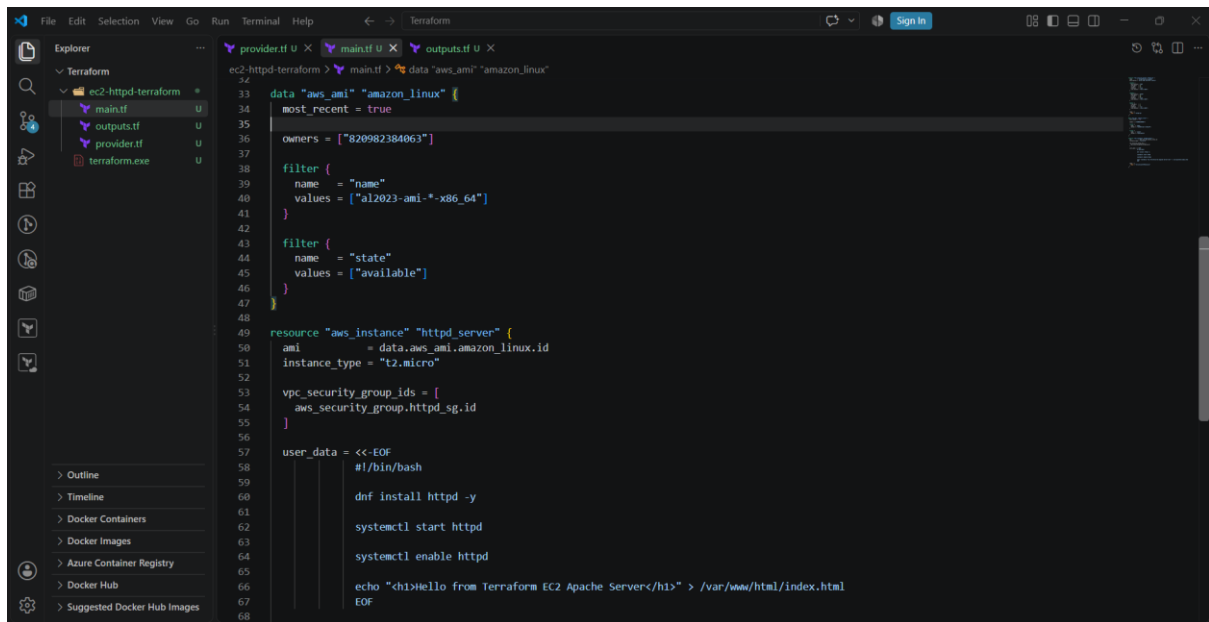
Put this in provider.tf:



Step 3: Create main.tf

Put this in main.tf:

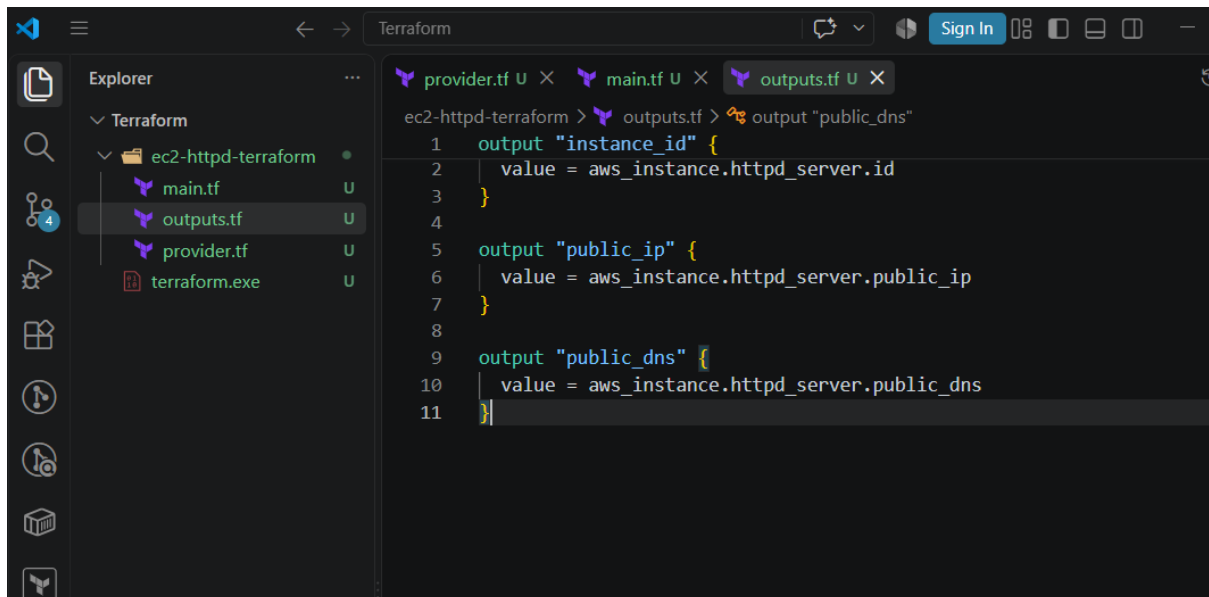




```
33 data "aws_ami" "amazon_linux" {
34   most_recent = true
35
36   owners = ["820982384063"]
37
38   filter {
39     name   = "name"
40     values = ["al2023-ami-*x86_64"]
41   }
42
43   filter {
44     name   = "state"
45     values = ["available"]
46   }
47 }
48
49 resource "aws_instance" "httpd_server" {
50   ami           = data.aws_ami.amazon_linux.id
51   instance_type = "t2.micro"
52
53   vpc_security_group_ids = [
54     aws_security_group.httpd_sg.id
55   ]
56
57   user_data = <<-EOF
58   #!/bin/bash
59
60   dnf install httpd -y
61
62   systemctl start httpd
63
64   systemctl enable httpd
65
66   echo "<h1>Hello from Terraform EC2 Apache Server</h1>" > /var/www/html/index.html
67   EOF
68 }
```

Step 5: Create outputs.tf

Put this in outputs.tf:



```
1 output "instance_id" {
2   value = aws_instance.httpd_server.id
3 }
4
5 output "public_ip" {
6   value = aws_instance.httpd_server.public_ip
7 }
8
9 output "public_dns" {
10  value = aws_instance.httpd_server.public_dns
11 }
```

Step 6: Configure AWS credentials

Before Terraform can create the EC2 instance, your AWS CLI must be configured with the correct account.

Check:

```
aws sts get-caller-identity
```

Make sure the account shown is your **new AWS account**.

Also check:

aws configure list

You should see your configured region and credentials.

Step 7: Initialize Terraform

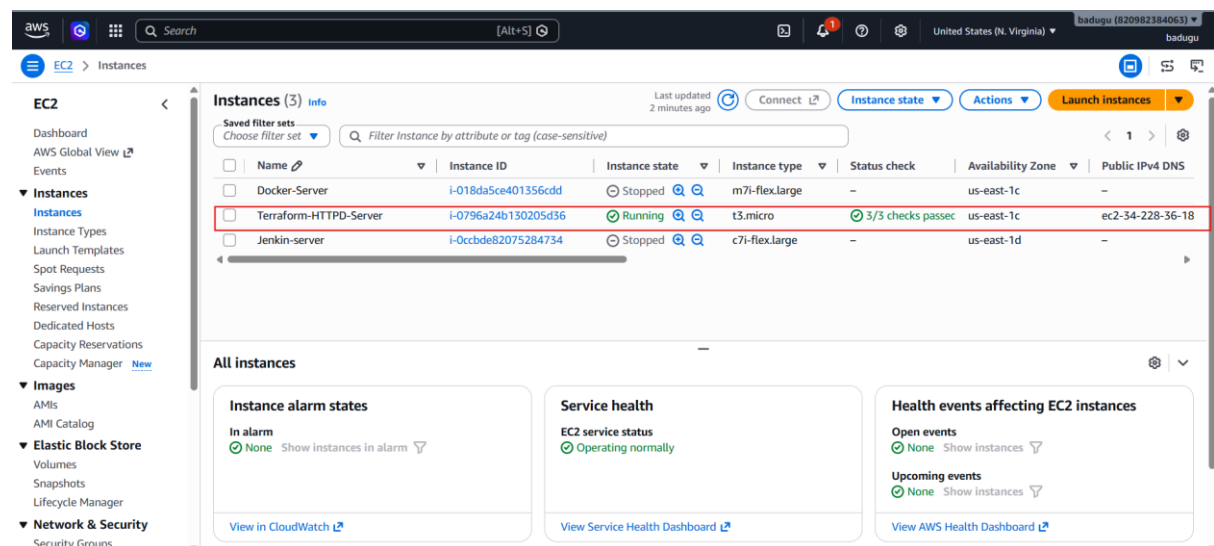
Open the terminal inside your Terraform folder.

Run:

terraform init

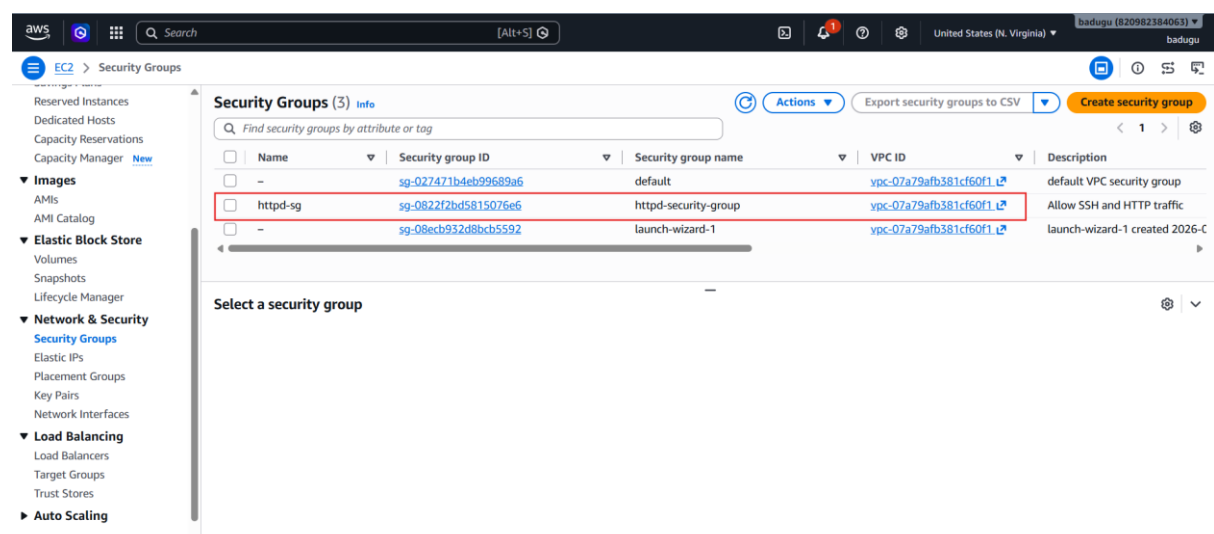
You should see something similar to:

Initializing the backend...



The screenshot shows the AWS Management Console for the 'United States (N. Virginia)' region. The 'EC2' service is selected, and the 'Instances' page is displayed. A table lists three instances: 'Docker-Server' (Stopped), 'Terraform-HTTPD-Server' (Running), and 'Jenkin-server' (Stopped). The 'Terraform-HTTPD-Server' instance is highlighted with a red border. Below the table, there are sections for 'All instances', 'Instance alarm states', 'Service health', and 'Health events affecting EC2 instances'.

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS
Docker-Server	i-018da5ce401356cdd	Stopped	m7i-flex.large	–	us-east-1c	–
Terraform-HTTPD-Server	i-0796a24b130205d36	Running	t3.micro	3/3 checks passed	us-east-1c	ec2-34-228-36-18
Jenkin-server	i-0ccbbe82075284734	Stopped	c7i-flex.large	–	us-east-1d	–



The screenshot shows the AWS Management Console for the 'United States (N. Virginia)' region. The 'EC2' service is selected, and the 'Security Groups' page is displayed. A table lists three security groups: 'default', 'httpd-sg', and 'launch-wizard-1'. The 'httpd-sg' security group is highlighted with a red border. Below the table, there is a section for 'Select a security group'.

Name	Security group ID	Security group name	VPC ID	Description
–	sg-027471b4eb99689a6	default	ypc-07a79afb381cf60f1	default VPC security group
httpd-sg	sg-0822f2bd5815076e6	httpd-security-group	ypc-07a79afb381cf60f1	Allow SSH and HTTP traffic
–	sg-08ecb932d8bcb5592	launch-wizard-1	ypc-07a79afb381cf60f1	launch-wizard-1 created 2026-C

3.Set up S3 as backend for task 3.

For **Task 3**, you can configure **Amazon S3 as the Terraform remote backend** so that terraform.tfstate is stored in S3 instead of only on your local computer.

Since you already created an S3 bucket in your new AWS account, let's do it step by step.

1. What are we going to configure?

Normally Terraform stores state locally:

Terraform



terraform.tfstate



Your local PC

With an S3 backend:

AWS



Terraform ————|



S3 Bucket



terraform.tfstate

This is useful because the state can be shared by a team and is not dependent on one developer's laptop.

Step 1: Make sure you're using the new AWS account

You already switched your CLI, but verify:

```
aws sts get-caller-identity
```

You should see:

Account: 820982384063

Step 2: Check your existing S3 bucket

You previously showed that this bucket exists:

terraform-jenkins-demo-123456789

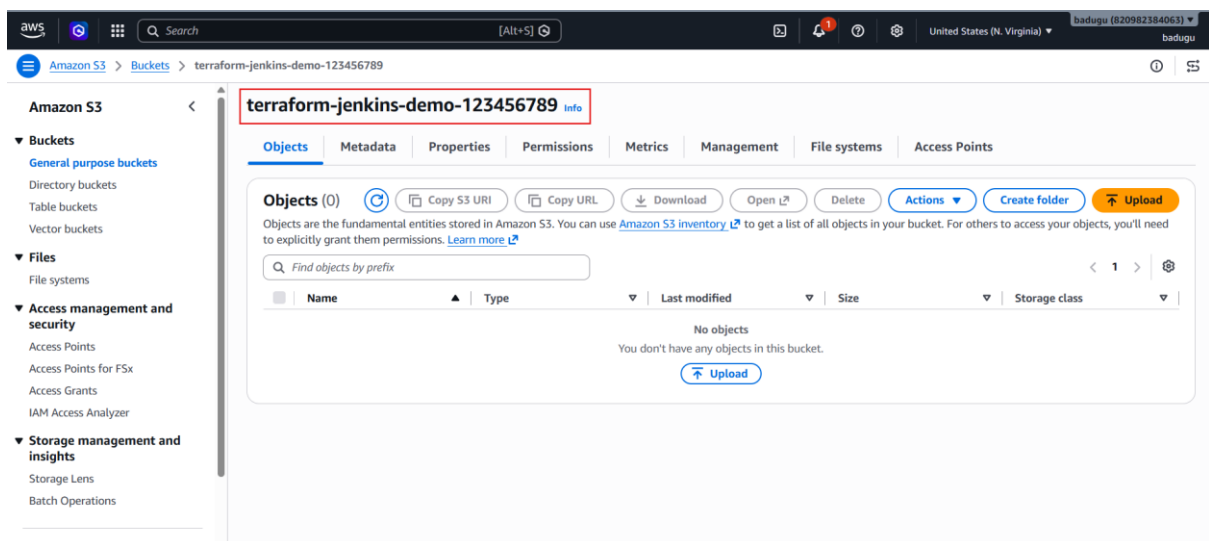
Check it:

```
aws s3 ls
```

You should see:

2026-09-02 19:33:36 terraform-jenkins-demo-123456789

We can use this bucket as the Terraform backend.



Step 3: Add the S3 backend to Terraform

In your Terraform project, open provider.tf.

Understand these three values

```
bucket = "terraform-jenkins-demo-123456789"
```

This is the S3 bucket where Terraform stores the state.

```
key = "task3/ec2-httpd/terraform.tfstate"
```

This is the **path/name of the state file inside the bucket.**

Your S3 bucket will look conceptually like:

terraform-jenkins-demo-123456789

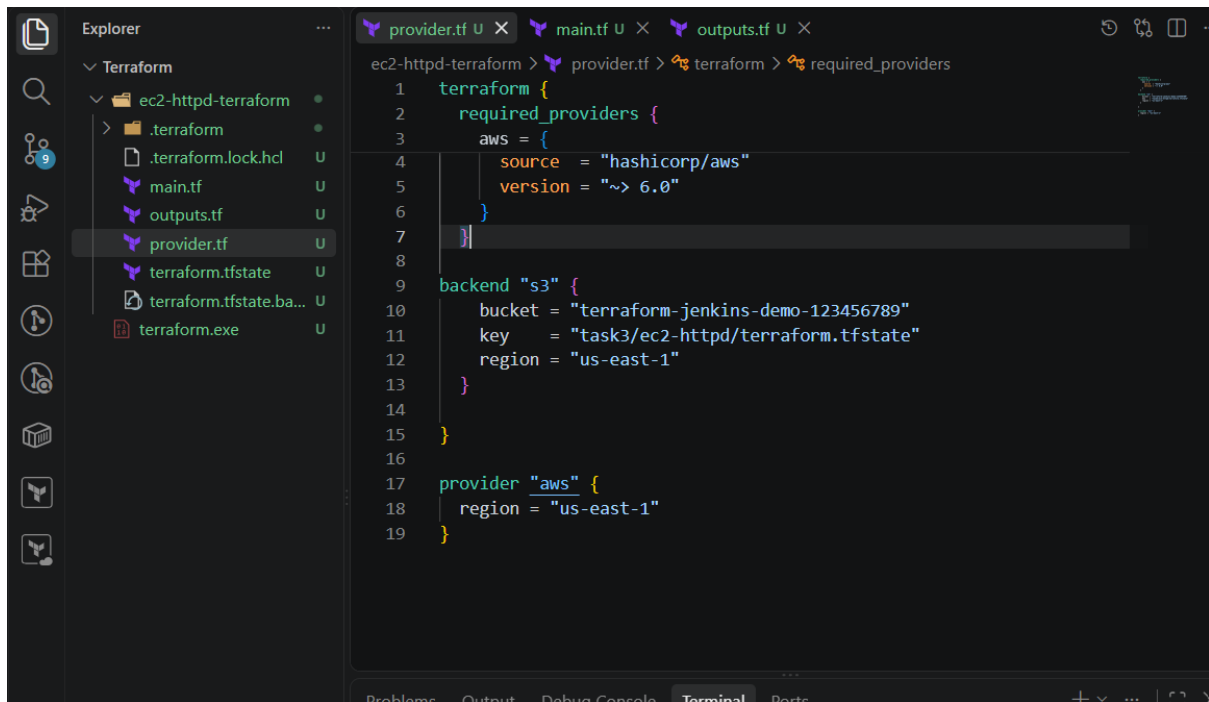
└── task3/

└── ec2-httpd/

└── terraform.tfstate

region = "us-east-1"

This is the region of the S3 bucket.



Step 4: Save the file

Your project should now look like:

ec2-httpd-terraform/

|

└── provider.tf

└── main.tf

└── outputs.tf

└── ...

Step 5: Initialize the S3 backend

Run:

terraform init

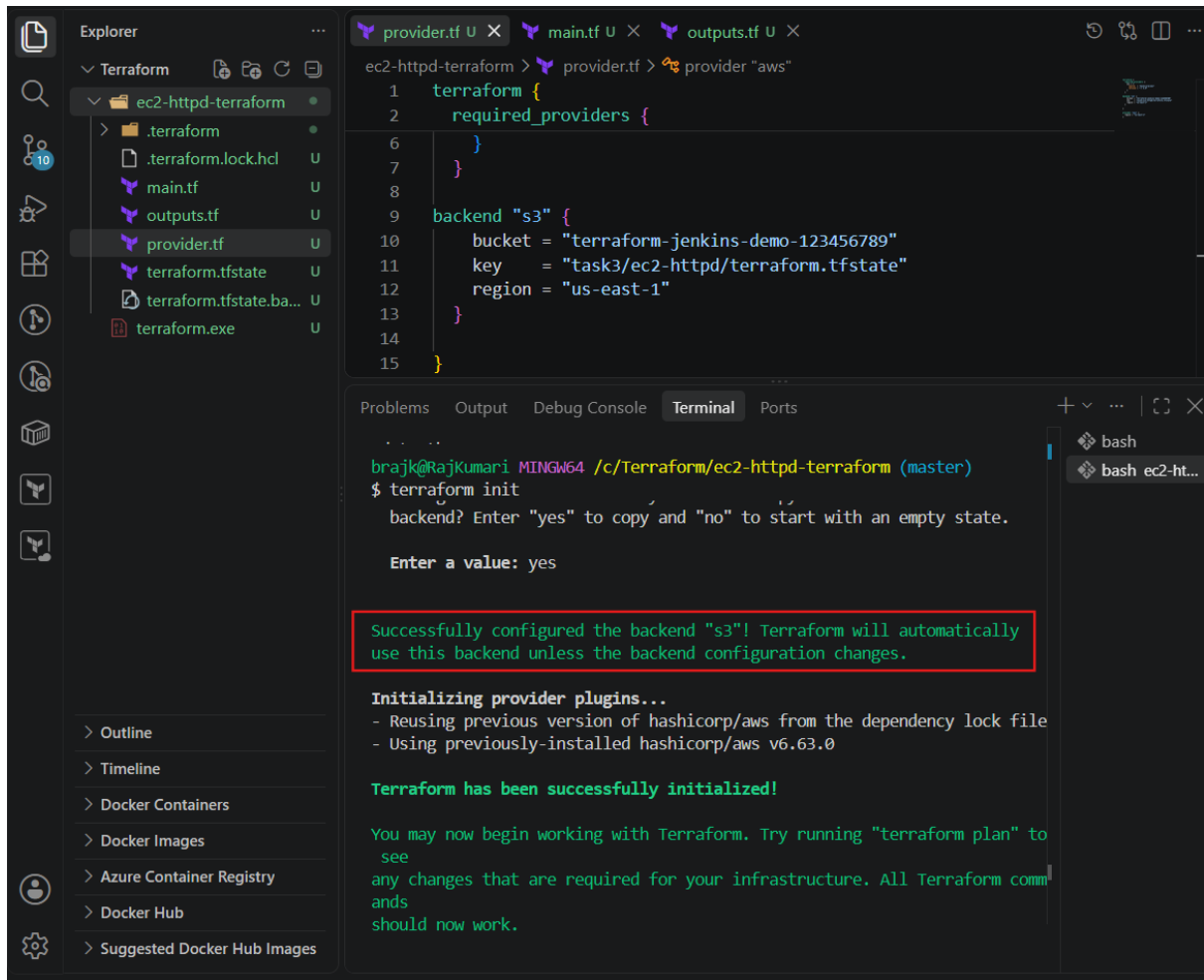
Because you changed the backend configuration, Terraform will detect the new backend.

You may see:

Initializing the backend...

and eventually:

Successfully configured the backend "s3"!



The screenshot shows a VS Code editor with a file explorer on the left and a terminal at the bottom. The file explorer shows a directory named 'ec2-httpd-terraform' containing files like '.terraform.lock.hcl', 'main.tf', 'outputs.tf', 'provider.tf', 'terraform.tfstate', and 'terraform.exe'. The 'provider.tf' file is open in the editor, showing a Terraform configuration with a backend 's3' block. The terminal shows the output of the 'terraform init' command, which includes a prompt to enter a value, a confirmation message, and a list of provider plugins.

```
provider.tf X main.tf U X outputs.tf U X
ec2-httpd-terraform > provider.tf > provider "aws"
1 terraform {
2   required_providers {
6   }
7 }
8
9 backend "s3" {
10   bucket = "terraform-jenkins-demo-123456789"
11   key    = "task3/ec2-httpd/terraform.tfstate"
12   region = "us-east-1"
13 }
14
15 }

Problems Output Debug Console Terminal Ports
brajk@Rajkumari MINGW64 /c/Terraform/ec2-httpd-terraform (master)
$ terraform init
  backend? Enter "yes" to copy and "no" to start with an empty state.

  Enter a value: yes

  Successfully configured the backend "s3"! Terraform will automatically
  use this backend unless the backend configuration changes.

  Initializing provider plugins...
  - Reusing previous version of hashicorp/aws from the dependency lock file
  - Using previously-installed hashicorp/aws v6.63.0

  Terraform has been successfully initialized!

  You may now begin working with Terraform. Try running "terraform plan" to
  see
  any changes that are required for your infrastructure. All Terraform comm
  ands
  should now work.
```

Step 6: If Terraform asks to migrate the state

Because you already ran terraform apply, you probably have a local:

terraform.tfstate

Terraform may ask:

Do you want to copy existing state to the new backend?

Choose:

yes

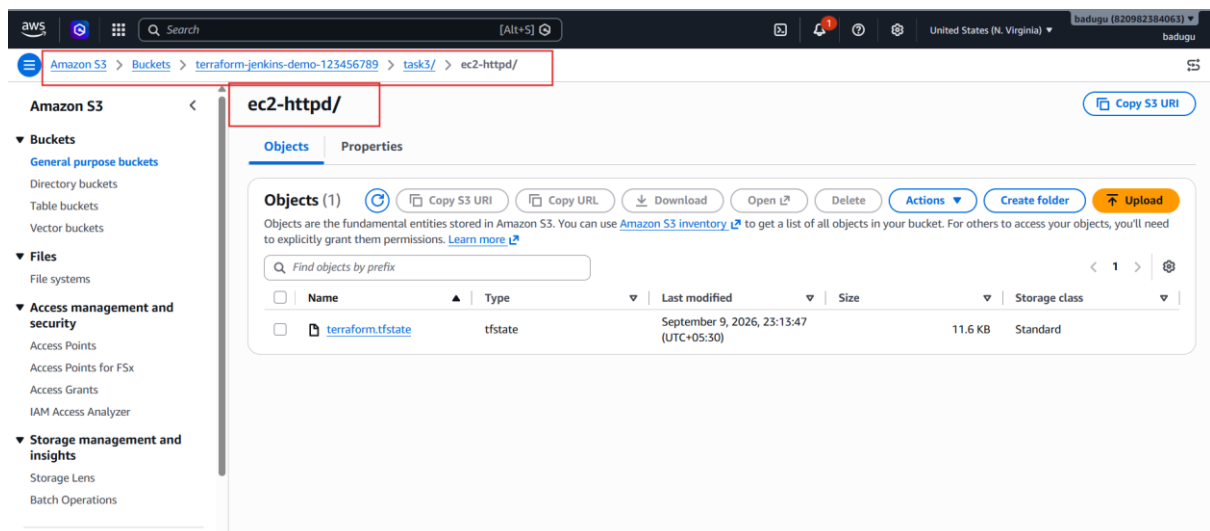
This is important.

Terraform will take your existing local state and upload it to:

S3

↓

task3/ec2-httpd/terraform.tfstate



Set up **DynamoDB locking** for task 3.

1. What is DynamoDB locking?

Without locking:

Developer A —┐

└───> terraform.tfstate

Developer B —┐

Both could run terraform apply at the same time, potentially causing state conflicts.

With locking:

Developer A

|

↓

DynamoDB Lock

|

↓

Terraform State in S3

|

↓

Developer B → WAIT

DynamoDB acts as a **lock mechanism**, while S3 stores the actual terraform.tfstate.

Step 1: Create the DynamoDB table

Use AWS CLI:

```
aws dynamodb create-table \  
  --table-name terraform-state-lock \  
  --attribute-definitions AttributeName=LockID,AttributeType=S \  
  --key-schema AttributeName=LockID,KeyType=HASH \  
  --billing-mode PAY_PER_REQUEST \  
  --region us-east-1
```

You should get a response showing:

TableName: terraform-state-lock

TableStatus: CREATING

Wait a few seconds.

Step 2: Verify the table

Run:

```
aws dynamodb describe-table \  
  --table-name terraform-state-lock \
```



```
--region us-east-1 \
```

```
--query "Table.{Name:TableName,Status:TableStatus}"
```

Expected:

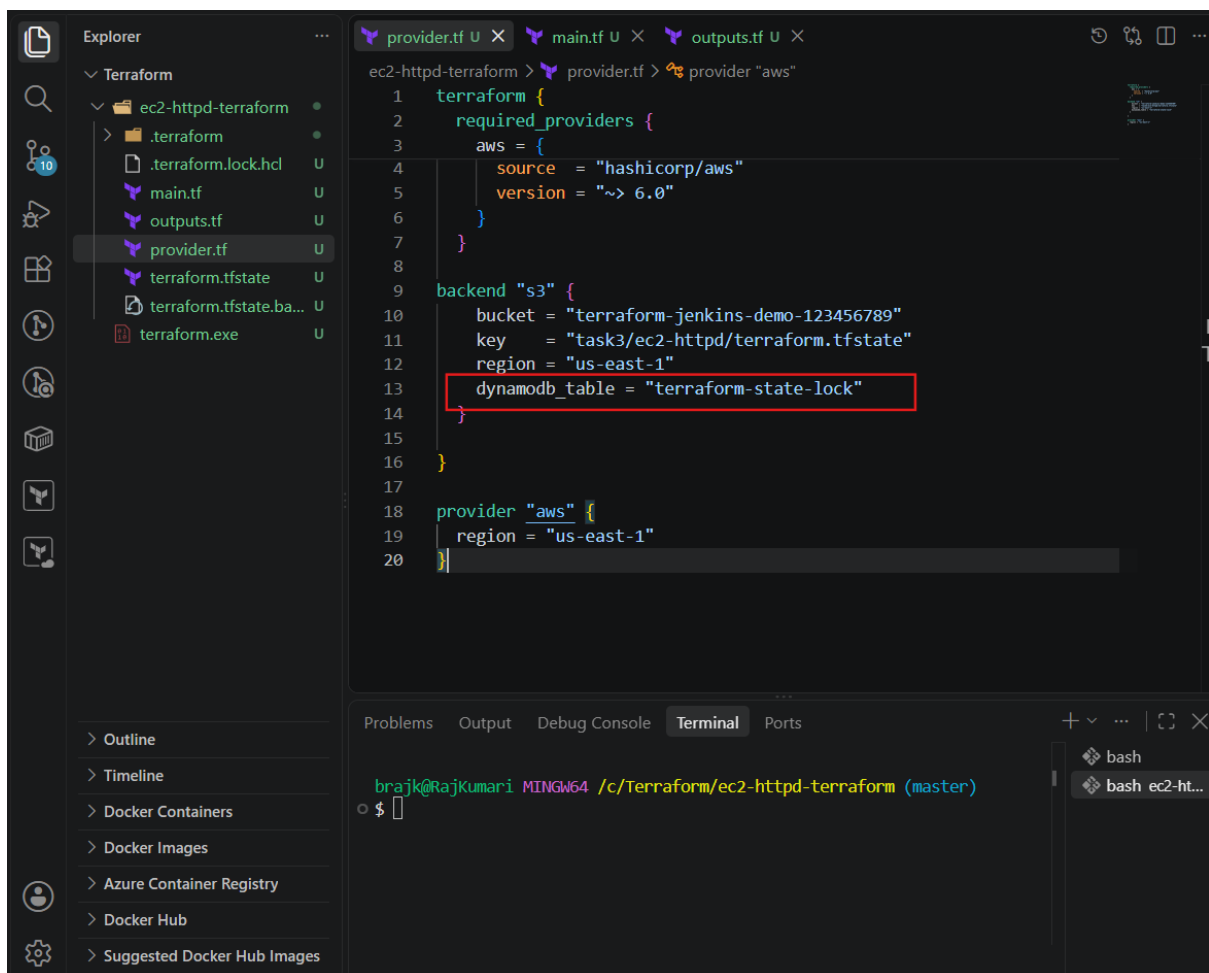
```
{
```

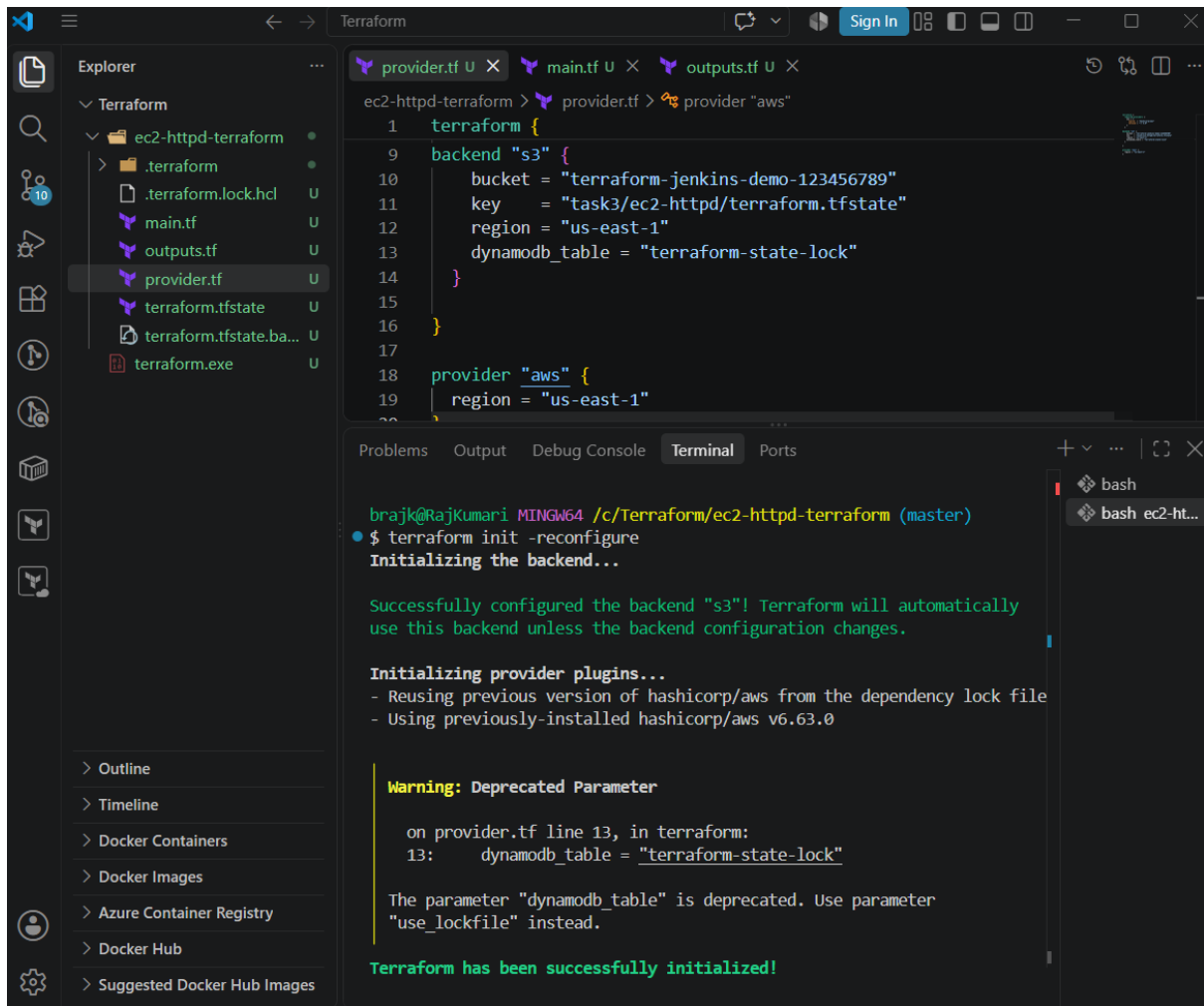
```
  "Name": "terraform-state-lock",
```

```
  "Status": "ACTIVE"
```

Step 3: Configure DynamoDB in provider.tf

You previously configured your S3 backend like this:





Step 5: Verify the backend

Run: terraform plan

Step 6: Understand what happens during terraform apply

When you run:

terraform apply

Terraform acquires a lock in DynamoDB before modifying the state.

Conceptually:

terraform apply

↓

Acquire DynamoDB lock

↓

Read terraform.tfstate from S3



Create/modify AWS resources



Update terraform.tfstate in S3



Release DynamoDB lock

The lock prevents another Terraform process from modifying the same state simultaneously.

S3 → stores Terraform state

DynamoDB → locks the state

The deprecation warning about dynamodb_table is separate from this error. For your assignment specifically asking for **DynamoDB locking**, you can continue with dynamodb_table.

